



Session 05:

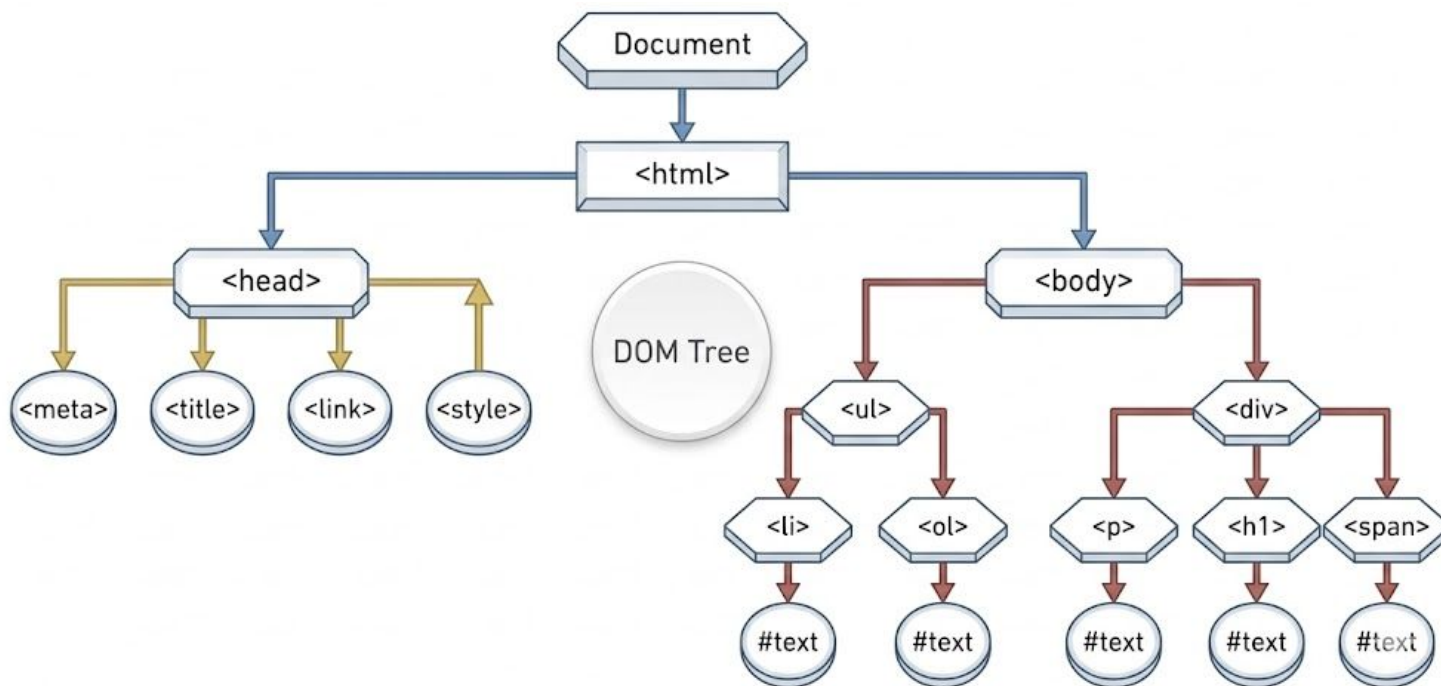
JavaScript DOM Fundamentals & Form Validation



NỘI DUNG

1. Biết, hiểu được về cấu trúc DOM
2. Hiểu, áp dụng được kỹ thuật thêm/xóa phần tử
3. Sử dụng được các phương thức gán, xử lý sự kiện
4. Áp dụng được kỹ thuật lấy, xác thực dữ liệu đầu vào

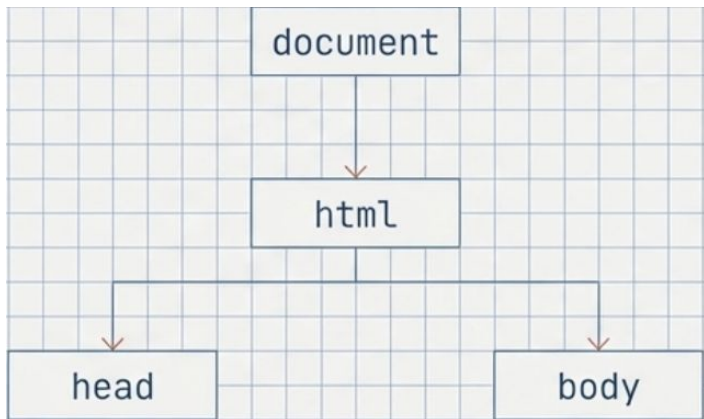
1. Tổng quan về DOM



Khái niệm DOM

DOM (**Document Object Model**) là mô hình đối tượng cho tài liệu HTML

- Trình duyệt chuyển HTML thành các 'node' (nút)
- Đây chính là cầu nối cho phép JavaScript đọc, thay đổi và tương tác với giao diện web



```
// Xem toàn bộ cấu trúc cây HTML  
console.log(document);
```

```
// Xem các thuộc tính hỗ trợ  
console.dir(document);
```

Truy xuất phần tử (cơ bản)

Truy xuất phần tử HTML cụ thể theo **ID**, **class** hoặc **tag name**

```
// Lấy một phần tử HTML duy nhất dựa trên thuộc tính id
document.getElementById("id");

// Lấy tất cả các phần tử HTML có cùng một tên lớp (class)
document.getElementsByClassName("class");

// Lấy tất cả các phần tử HTML có cùng tên thẻ (tag name)
document.getElementsByTagName("tag");
```

Cú pháp:

- `document.getElementById("id");`
- `document.getElementsByClassName("class");`
- `document.getElementsByTagName("tag");`

Truy xuất phần tử (CSS Selectors)

Truy xuất bằng **querySelector** và **querySelectorAll**

Cú pháp:

- `document.querySelector(".selector");`
- `document.querySelectorAll(".selector");`

```
// Lấy phần tử đầu tiên khớp với selector
let firstItem = document.querySelector(".item");

// Lấy tất cả các phần tử khớp với selector
let allItems = document.querySelectorAll(".item");
```

Thao tác nội dung văn bản

Thuộc tính	Cú pháp	Chức năng	Đặc điểm
innerText	<code>el.innerText = "Hello";</code>	Lấy hoặc thay đổi nội dung văn bản hiển thị mà người dùng nhìn thấy	Không lấy nội dung bị ẩn bởi display: none . Chậm hơn do phải tính toán layout
textContent	<code>el.textContent = "Hello";</code>	Lấy hoặc thay đổi tất cả nội dung văn bản bên trong node và các node con	Lấy hết chữ kể cả khi bị ẩn. Hiệu năng cao nhất, an toàn vì không thực thi thẻ HTML
innerHTML	<code>el.innerHTML = "Hello";</code>	Lấy hoặc thay đổi toàn bộ cấu trúc HTML bên trong phần tử	Trình duyệt sẽ render các thẻ HTML. Cần cẩn trọng vì dễ bị tấn công XSS (Cross-site Scripting)

Thao tác với Attribute (Thuộc tính)

Lấy, thay đổi hoặc thêm mới các **thuộc tính gốc** của thẻ HTML (như link ảnh, đường dẫn, giá trị ô nhập liệu)

Cú pháp:

- **Lấy giá trị:** `element.getAttribute("tên thuộc tính");`
- **Thêm, đổi giá trị:** `element.setAttribute("tên thuộc tính", "giá trị mới");`



```
<a id="link" href="google.com">Google</a>

```



```
const img = document.getElementById("logo");
// Đổi ảnh động
img.setAttribute("src", "new-logo.png");

const link = document.getElementById("link");
// Lấy url hiện tại
let url = link.getAttribute("href");
```


Điều chỉnh Style (CSS)

Thay đổi **CSS inline** trực tiếp trên phần tử bằng JavaScript thông qua thuộc tính **“.style”**

Cú pháp:

- `element.style.css_properties = “css_value”;`



```
const box = document.querySelector(".box");  
box.style.color = "red";  
box.backgroundColor = "yellow";
```

Quy tắc CamelCase

Trong JavaScript, tên thuộc tính CSS có dấu gạch ngang phải chuyển thành viết hoa chữ cái tiếp theo (CamelCase):

- background-color → **backgroundColor**
- font-size → **fontSize**
- border-radius → **borderRadius**

Quản lý class (classList)

Thuộc tính classList (**DOMTokenList**) quản lý danh sách các class CSS. Đây là cách hiện đại và dễ bảo trì nhất để thay đổi giao diện thay vì dùng “.style” trực tiếp

Cú pháp:

- .add(“class”): Thêm class mới
- .remove(“class”): Xóa class hiện có
- .toggle(“class”): Công tắc (Có thì xóa, chưa thì thêm)
- .contains(“class”): Kiểm tra tồn tại (trả về true/false)

```
let box = document.getElementById("box");

box.classList.add("active");
box.classList.remove("big");

// Hoàn hảo cho nút bấm Menu hoặc Dark Mode
box.classList.toggle("highlight");

// Trả về true / false
let isRed = box.classList.contains("red");
```

DOM Traversal - Duyệt cây node (Cha & Con)

Duyệt cây Node (**DOM Traversal**) cho phép di chuyển lên/xuống giữa các phần tử có quan hệ huyết thống mà không cần dùng “**querySelector**” tìm lại từ đầu

Cú pháp:

- Node Cha: `element.parentElement`
- Node Con `element.children` (chỉ lấy thẻ HTML), `firstElementChild`, `lastElementChild`

```
const list = document.getElementById("unordered-list");

// Lấy <li> đầu tiên
const firstItem = list.firstElementChild;

// Di chuyển ngược từ con lên cha
const parent = firstItem.parentElement;

// Lấy danh sách toàn bộ thẻ con
const allItems = list.children;
```

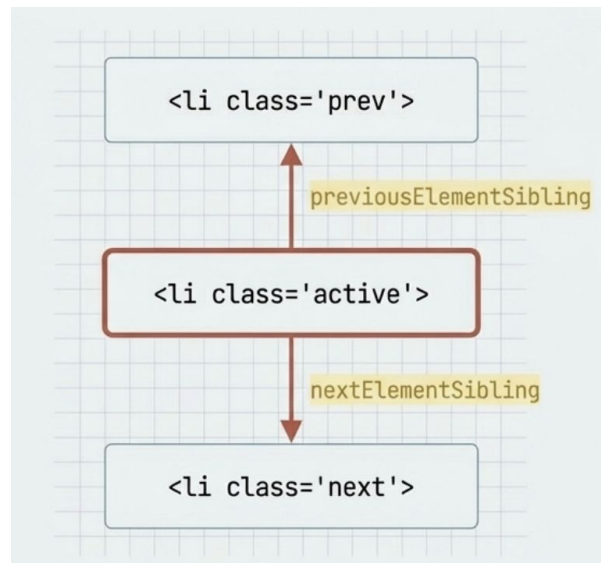
DOM Traversal - Duyệt cây node (Anh & Em)

Truy cập các phần tử liên kề trong cùng một thẻ cha

Cú pháp:

- Node kế tiếp: `element.nextElementSibling`
- Node phía trước: `element.previousElementSibling`

```
const current = document.querySelector("active");  
  
// Chuyển sang thẻ bên cạnh  
const nextItem = current.nextElementSibling;  
  
// Quay lại thẻ phía trước  
const prevItem = current.previousElementSibling;
```



Thêm phần tử vào DOM

1. Tạo (Create)

Tạo phần tử mới hoàn toàn bằng bộ nhớ RAM

```
const newLi = document.  
createElement("li");
```

2. Cấu hình (Configure)

Gán nội dung, thêm class, hoặc gán sự kiện

```
newLi.textContent =  
"New item";
```

3. Chèn (Append)

Gắn phần tử từ bộ nhớ vào cấu trúc trang web để hiển thị

```
document.getElementById(  
"list").appendChild(  
newLi);
```

Xóa phần tử khỏi DOM

Gỡ bỏ hoàn toàn 1 phần tử đang hiển thị ra khỏi cấu trúc trang web

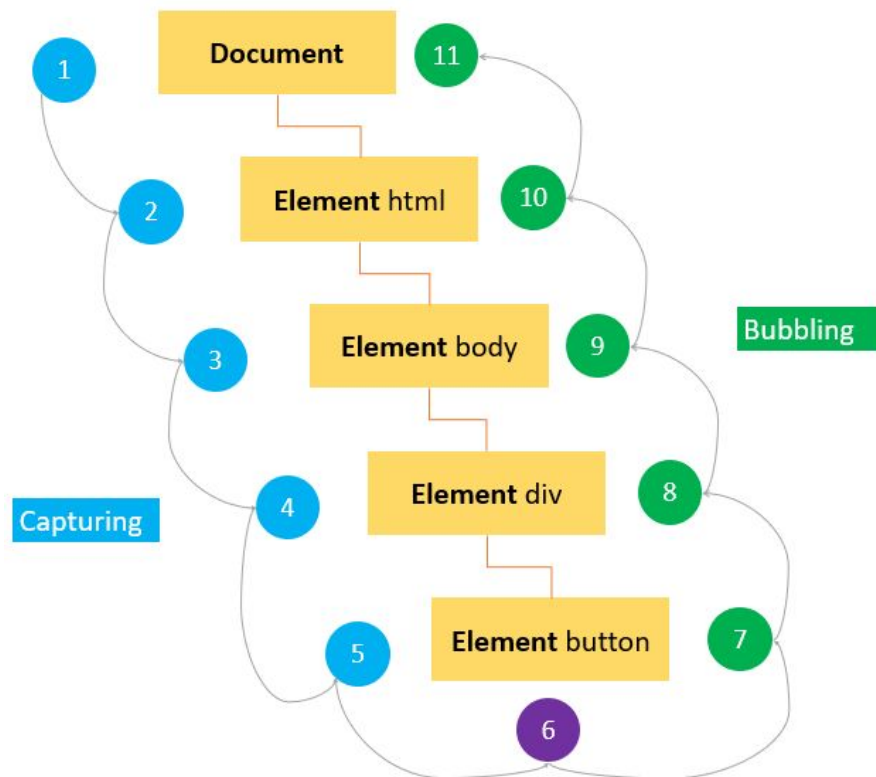
Cú pháp:

- Cách hiện đại: `element.remove()`
- Cách cũ (Trình duyệt cũ): `parentNode.removeChild(childNode)`

```

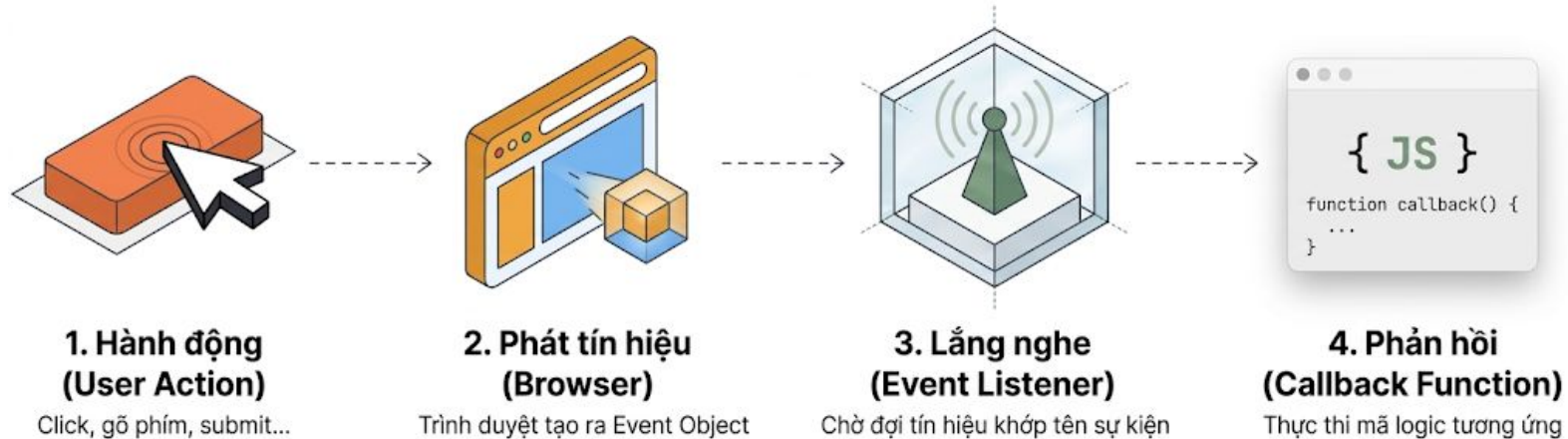
// Cách 1: Hiện đại, ngắn gọn
const banner = document.getElementById("ads-banner");
banner.remove();
// Cách 2: Thông qua thẻ cha
const list = document.getElementById("myList");
const item = document.getElementById("item-1");
list.removeChild(item);
```

2. DOM Event



Khái niệm Event

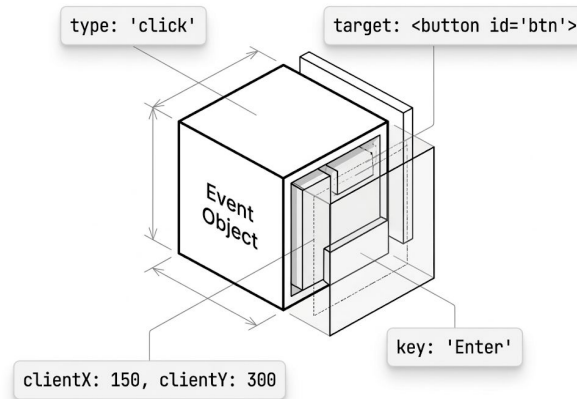
Sự kiện (event) là các hành động xảy ra trong trình duyệt. Event Listener là cầu nối giúp trang web **lắng nghe** và **thực thi** mã JavaScript để phản hồi lại các hành động đó



Event Object

Event Object là một đối tượng chứa toàn bộ thông tin chi tiết về sự kiện vừa xảy ra do trình duyệt tự động đóng gói

- Hàm callback luôn tự động nhận tham số này ở vị trí đầu tiên
- Thường được viết tắt là “e” hoặc “event”



Các loại sự kiện phổ biến

Nhóm sự kiện	Ví dụ	Ý nghĩa
Chuột	click, dblclick, mouseover	Nhấp, rê, di chuột
Bàn phím	keydown, keyup, keypress	Gõ hoặc thả phím
Form	submit, change	Gửi biểu mẫu, thay đổi giá trị input
Nhập liệu	input	Người dùng nhập vào một ô
Tài liệu / Trình duyệt	load, scroll, resize	Trang tải xong, cuộn trang, thay đổi kích thước

Gắn sự kiện nội tuyến (Inline Event)

Gắn trực tiếp lệnh **JavaScript** và thẻ **HTML** thông qua các thuộc tính bắt đầu bằng “**on**” như (“**onclick**”, “**onmouseover**”)

- **Ưu điểm:** Viết rất nhanh gọn
- **Nhược điểm:** Mã JavaScript và HTML bị trộn lẫn khó bảo trì. Không thể gắn nhiều hành động cho cùng 1 sự kiện

```
<button onclick="handleClick()">Nhấn vào tôi</button>
<script>
    function handleClick() {
        alert("Bạn đã nhấp vào nút!");
    }
</script>
```

Event Handler Properties

Thao tác gắn hàm xử lý trực tiếp lên thuộc tính sự kiện của đối tượng DOM trong file JavaScript

- **Ưu điểm:** Tách biệt hoàn toàn mã JavaScript và HTML
- **Nhược điểm:** Chỉ gắn được 1 hàm duy nhất. Nếu gắn lại, hành động cũ sẽ bị ghi đè hoàn toàn mà không có cảnh báo

```
<button id="btn2">Nhấn vào tôi</button>
<script>
  const btn2 = document.getElementById("btn2");
  btn2.onclick = function() {
    alert("Nút thứ 2 đã được nhấn!");
  }
  // Ghi đè, hành động cũ bị xóa
  btn2.onclick = function() {
    alert("Ghi đè!");
  }
</script>
```

addEventListener

Phương thức tiêu chuẩn, mạnh mẽ nhất để đăng ký sự kiện trong JavaScript

- **Ưu điểm:**
 - Không bao giờ ghi đè, gán được nhiều hàm cho cùng 1 sự kiện
 - Tách biệt HTML và JavaScript hoàn toàn
 - Hỗ trợ gỡ bỏ sự kiện bằng removeEventListener()

```

<button id="btn3">Click me!</button>
<script>
  const btn3 = document.getElementById("btn3");
  btn3.addEventListener("click", function () {
    alert("Nút thứ 3 đã được click!");
  });
</script>
```

So sánh 3 phương pháp gắn sự kiện

Tiêu chí	Inline Events	DOM Properties (on...)	addEventListener
Vị trí viết code	Viết trong HTML	Viết trong JS	Viết trong JS
Tách biệt HTML & JS	✗ Không	⚠ Một phần	✓ Hoàn toàn
Gắn nhiều sự kiện	✗ Không	✗ Bị ghi đè	✓ Có (Stacking)
Sự kiện hiện đại	✗ Hạn chế	✗ Hạn chế	✓ Đầy đủ
Khuyến nghị	✗ Bỏ qua	⚠ Tạm ổn	★ Tiêu chuẩn (Khuyến dùng)

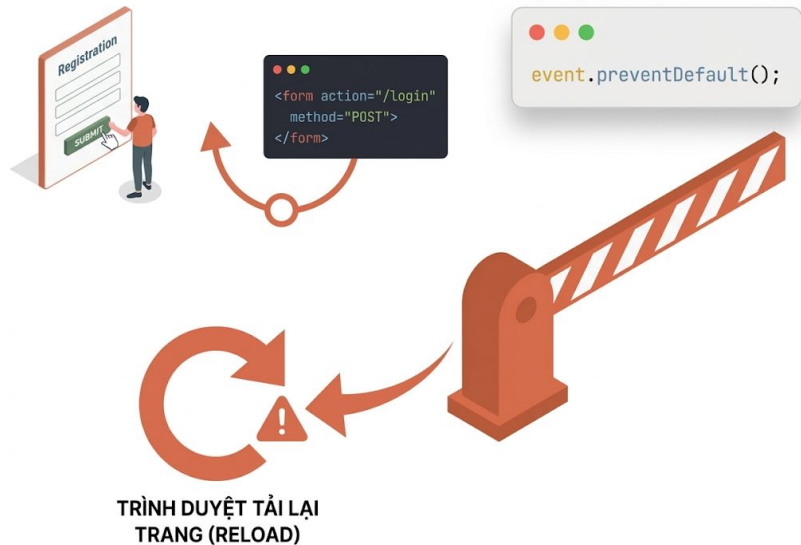
Sự kiện Form & gửi hành vi mặc định

- **Bản chất:** Khi bấm Submit hoặc Enter, form thu thập dữ liệu và tải lại toàn bộ trang web theo mặc định
- **Vấn đề:** Gây gián đoạn trải nghiệm người dùng, mất trạng thái UI và không thể kiểm tra lỗi bằng JavaScript trước khi gửi



Kiểm soát luồng dữ liệu với `preventDefault()`

Phương thức **`PreventDefault()`** hủy sự kiện nếu nó có thể hủy được, nghĩa là hành động mặc định thuộc về sự kiện sẽ không xảy ra



```
const form = document.getElementById("form");
form.addEventListener("submit", function(event) {
  // Ngăn chặn hành vi mặc định của form
  event.preventDefault();
})
```


Trích xuất dữ liệu từ Form

Gắn thuộc tính “**name**” cho thẻ HTML là chìa khóa để trích xuất dữ liệu nhanh chóng thông qua đối tượng form

focus
(Được chọn)

input
(Kích hoạt liên tục mỗi khi gõ)

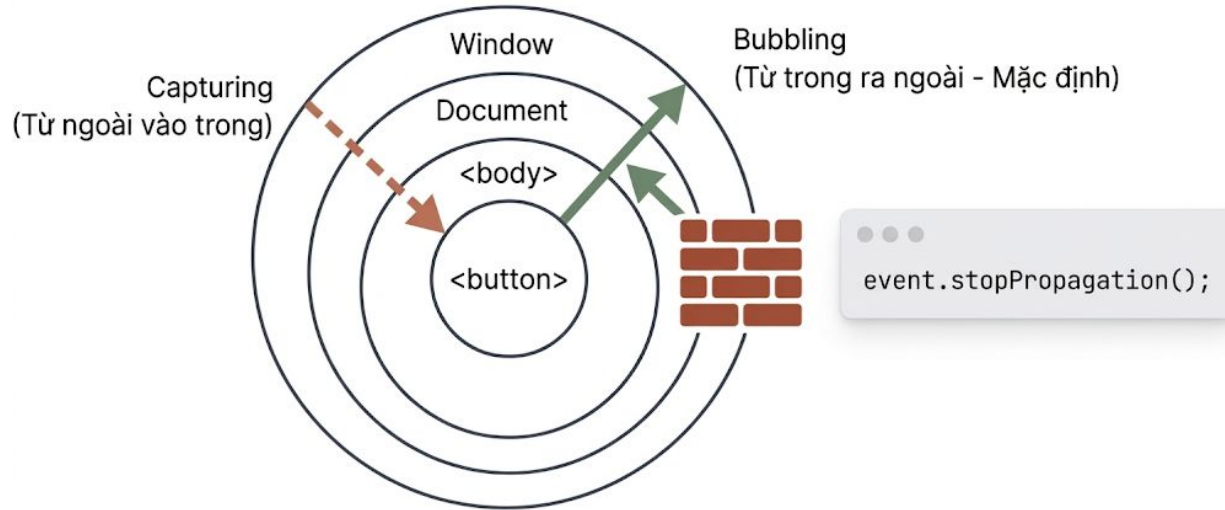
blur / change
(Rời khỏi và giá trị đã đổi)

```
<input type="email" name="email">  
<input type="password" name="password">
```

```
const email = form.email.value;  
const password = form.password.value;
```

Dòng chảy sự kiện (Event Flow)

Sự kiện không chỉ xảy ra trên một phần tử, nó lan truyền qua cấu trúc DOM. Dùng `stopPropagation()` để 'cách ly' sự kiện, ngăn việc 'click' từ phần tử con kích hoạt sự kiện phần tử cha



- ❑ Hiểu mô hình DOM và cách tương tác với giao diện web
- ❑ Nắm vững các phương thức truy xuất phần tử
- ❑ Phân biệt được cách thao tác nội dung qua innerText, textContent và innerHTML
- ❑ Biết cách thay đổi thuộc tính (Attribute), style CSS (.style) và quản lý class (classList)
- ❑ Thành thạo việc thao tác thêm/xóa phần tử khỏi cấu trúc trang web
- ❑ Hiểu bản chất của Event và Event Object
- ❑ Phân biệt 3 phương pháp gắn sự kiện
- ❑ Nắm được cách trích xuất dữ liệu từ Form



HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN